

## Deallocation Memory

Whenever we declare a variable in our code, the computer has to physically block off a tiny piece of its RAM to store that data. This process is called **Allocation**. When you are done using that data, the computer needs to free up that space so it can be used again. This is called **Deallocation**.

### 1. The Danger of Memory Leaks

Imagine you write a script that constantly runs in the background. If that script allocates new memory space every single time it executes, but never deallocates the space when it's done, it will eventually eat up all the system's RAM and crash the system.

This scenario is called a **Memory Leak**. To prevent this, memory must be deallocated in a timely fashion.

### 2. The Two Realms of Memory: Stack vs. Heap

To manage allocation and deallocation efficiently, the computer divides its memory into two main areas:

- **The Stack (Temporary Memory):** This area is strictly dedicated to **function calls** and their **local variables**.
  - **How it works:** When a function runs, its variables are placed on the Stack. The moment the function finishes, the stack automatically destroys those variables and deallocates the space.
- **The Heap (Persistent Memory):** This area is for data that needs to **live longer** than the function that created it.
  - **How it works:** If you put a variable on the Heap, it stays there permanently until it is **explicitly deallocated**. It does not automatically go away when a function ends.

**NOTE:** Go does something incredibly clever behind the scenes with the **Stack** and **Heap**, which we will see later

### 3. The "Old Way" of Deallocation (Manual Management)

In older compiled languages like **C** or **C++**, the programmer is **100%** responsible for the Heap.

- If you want memory, you have to write a command like **malloc( )** (Memory Allocate) to grab space on the **Heap**.
- When you are done, you must remember to write **free( )** to deallocate it.
- **The Problem:** It is incredibly fast, but highly **error-prone**. If you forget to write **free( )**, you get a **memory leak**. If you write **free( )** too early, your **program crashes** because it tries to access data that no longer exists.

#### Expert Takeaway:

This is why Go is so beloved in backend architecture. Interpreted languages (like **Python** or **JavaScript**) handle the Heap for you automatically, but they are slow. Languages like **C** and **C++** are blazing fast, but force you to manually juggle **malloc** and **free**.

Go gives you the raw speed of C, but it uses a built-in "**Garbage Collector**" that acts as an automated street sweeper for the Heap, silently freeing memory you no longer need so you never have to write it yourself.